

Incremental Crawlers in Information Retrieval

1. Introduction

A *web crawler* is a software program that automatically traverses the web, downloads pages, and extracts information. In standard crawling, a crawler repeatedly downloads all pages irrespective of changes. This leads to:

- High bandwidth consumption
- Outdated indexes
- Slow update cycles
- Redundant processing

To solve this problem, search engines use **Incremental Crawlers**, which update only those pages that have changed since the last crawl. Their main goal is **freshness, efficiency, and low overhead** in large-scale IR systems.

2. What Are Incremental Crawlers?

Incremental crawlers are specialized crawlers designed to:

- Detect updates on previously fetched pages
- Re-crawl only changed or new pages
- Maintain fresh indexes with minimal resource usage

Instead of downloading full pages repeatedly, incremental crawlers use intelligent mechanisms like:

- **Hash comparison**
- **Content difference analysis**
- **HTTP metadata (ETag, Last-Modified)**
- **Machine learning–based change prediction**

They are essential for rapidly changing websites such as:

- News portals
- E-commerce sites
- Social media feeds
- Blogs and forums

3. Need for Incremental Crawling

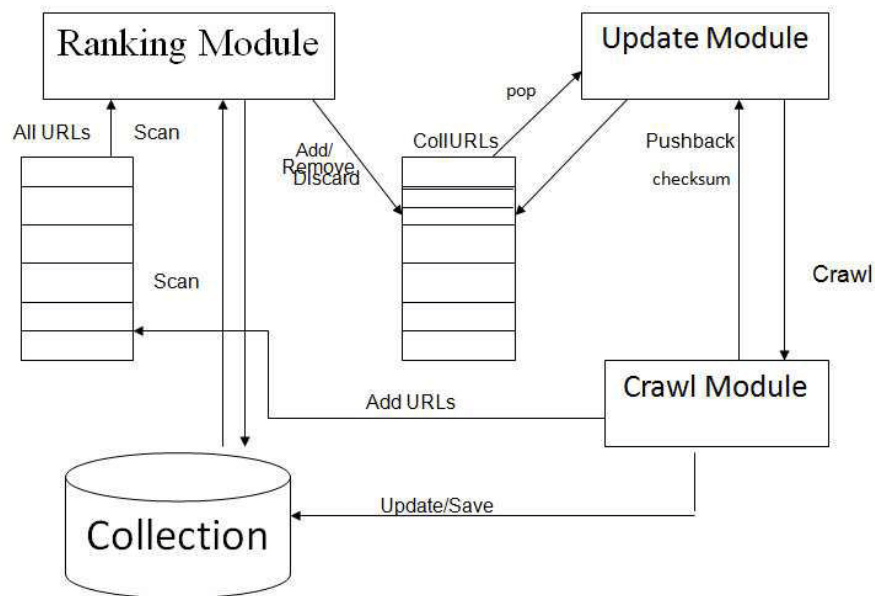
Limitation of Traditional Crawlers	How Incremental Crawler Solves It
Downloading same page many times	Re-download only if changed

Limitation of Traditional Crawlers	How Incremental Crawler Solves It
High bandwidth & processing cost	Efficient recrawl scheduling
Slow index update	Faster, targeted updates
Outdated search results	Improved freshness

Incremental crawling ensures users always get the **most recent version** of the web.

4. Architecture of Incremental Crawlers

The architecture is designed to detect changes, prioritize revisits, and update indexes with minimal overhead. The key components are:



5. Working Procedure of an Incremental Crawler

The workflow of an incremental crawler can be explained in the following steps:

Step 1: Initialization

- Start with a list of seed URLs
- Assign change probabilities
- Initialize URL Frontier

Step 2: Scheduling Recrawls

The scheduler assigns the next visit time using:

- Averaged change frequency

- Predicted changes
- Page importance

Example:

Highly dynamic pages → higher priority.

Step 3: Fetching the Page

The fetcher:

- Sends HTTP HEAD to check for update
- If changed, sends GET request
- If "Not Modified", skips downloading

This drastically reduces bandwidth.

Step 4: Change Detection

After fetching:

- Compare current hash with stored hash
- Analyze ETag/Last-Modified headers
- Perform structural diff if needed

If unchanged → move to next URL.

Step 5: Parsing & Analysis

If changed:

- Extract updated content
- Identify new links
- Identify deleted content
- Pass data to Update Analyzer

Step 6: Repository Update

Update repository with:

- New page version
- Updated metadata
- New timestamp
- Updated hash

Old versions may be kept for history.

Step 7: Incremental Indexing

Only changed parts are re-indexed,
This reduces:

- CPU usage
- Memory usage
- Indexing time

Step 8: Update URL Frontier

New or updated outgoing links are added to URL Frontier with appropriate priority.

Step 9: Loop Continues

The cycle repeats indefinitely to maintain fresh index.

6. Advantages of Incremental Crawlers

- ✓ Efficient use of bandwidth
- ✓ Improved freshness of index
- ✓ Faster crawling cycles
- ✓ Scalable for large systems
- ✓ Lower storage and CPU overhead
- ✓ Helps maintain real-time search results

7. Applications of Incremental Crawlers

- Major search engines (Google, Bing)
- News aggregators
- Social media sites
- E-commerce price monitoring
- Stock market trackers
- Digital library updates
- Scientific data monitoring

PageRank and HITS in Information Retrieval

1. Introduction

Modern search engines must decide *which pages are important* among millions of web pages. Two of the most famous link-analysis algorithms used in IR are:

1. **PageRank** – by Google
2. **HITS (Hyperlink-Induced Topic Search)** – by Jon Kleinberg

Both rely on *link structures* among web pages to compute **importance**, but each does so differently.

2. PageRank Algorithm

PageRank assigns a **global importance score** to each webpage based on:

- The number of incoming links
- The quality (importance) of the linking pages
- The idea that *links are votes of confidence*

2.1 Intuition

A page is important if:

- Many pages link to it
- Or a few highly important pages link to it

This is a recursive definition.

2.2 PageRank Formula

For a page A :

$$PR(A) = \frac{1 - d}{N} + d \sum_{i \in In(A)} \frac{PR(i)}{Out(i)}$$

Where:

- d = damping factor (~ 0.85)
- N = total number of pages
- $In(A)$ = pages linking to A
- $Out(i)$ = number of outgoing links from page i

2.3 Steps in PageRank

1. Initialize all pages with equal rank
2. Iteratively update PageRank based on incoming links
3. Continue until convergence

4 Example: Small Web Graph

Consider 3 pages: A, B, C

- $A \rightarrow B$

- $B \rightarrow C$
- $C \rightarrow A$

Each page receives one link.

Thus PageRank values eventually stabilize equally:

$$PR(A) = PR(B) = PR(C)$$

This is because the link structure is perfectly circular.

3. HITS Algorithm (Hyperlink-Induced Topic Search)

Unlike PageRank, HITS divides importance into **two concepts**:

- **Authority Score:** how useful a page's content is
- **Hub Score:** how good a page is at pointing to relevant authorities

3.1 Two Roles in HITS

Authority:

A page that *many good hubs point to*.

Example:

A page containing high-quality information (Wikipedia page).

Hub:

A page that *links to many good authorities*.

Example:

A curated list of resources ("Best AI Resources").

3.2 HITS Algorithm Steps

1. Construct a **root set** (pages related to query)
2. Expand into a **base set** by adding linked pages
3. Initialize all authority and hub scores = 1
4. Update iteratively:
 - Authority score = sum of hub scores of pages linking in
 - Hub score = sum of authority scores of pages it links to
5. Normalize scores
6. Repeat until convergence

4. Example of HITS With "Hit & Miss" (Hubs & Authorities)

Consider 4 pages:

- **P1** links to P3
- **P2** links to P3
- **P3** links to P4
- **P4** links to no one

Link Structure:

- $P1 \rightarrow P3$
- $P2 \rightarrow P3$
- $P3 \rightarrow P4$

4.1 Initial Scores

All pages start as:

Page Authority Hub

P1	1	1
P2	1	1
P3	1	1
P4	1	1

4.2 First Update

Authority Scores

- P3 gets authority from P1 and P2:
→ $\text{Authority}(P3) = \text{Hub}(P1) + \text{Hub}(P2) = 1 + 1 = 2$
- P4 gets authority from P3:
→ $\text{Authority}(P4) = \text{Hub}(P3) = 1$
- P1, P2 have no inbound links, so:
→ $\text{Authority}(P1) = \text{Authority}(P2) = 0$

Hub Scores

- P1 links to P3:
→ $\text{Hub}(P1) = \text{Authority}(P3) = 2$
- P2 links to P3:
→ $\text{Hub}(P2) = \text{Authority}(P3) = 2$
- P3 links to P4:
→ $\text{Hub}(P3) = \text{Authority}(P4) = 1$
- P4 has no outbound links:
→ $\text{Hub}(P4) = 0$

4.3 After Normalization (Simplified)

Page Authority Hub

P1	0	2
P2	0	2
P3	2	1
P4	1	0

4.4 Hit & Miss Explanation

In HITS:

- A **Hit** means a page scores well as a **hub** or an **authority**.
- A **Miss** means a page fails to gain importance (gets near-zero values).

In our example:

Hits

- **P3 is a HIT Authority**
Because many good hubs (P1, P2) point to it.
- **P1 & P2 are HIT Hubs**
They point to a good authority (P3).

Misses

- **P1 & P2 are Miss Authorities**
No one points to them.
- **P4 is a Miss Hub**
It has no outgoing links.

Page	HIT / MISS	Reason
P1	HIT Hub	Points to strong authority (P3)
P2	HIT Hub	Same as P1
P3	HIT Authority	Receives strong hub links
P4	MISS Hub	No outbound links

Page HIT / MISS Reason

P1,P2 MISS Authority No inbound links

4. Difference Between PageRank and HITS

Feature	PageRank	HITS
Output	One score per page	Two scores (hub & authority)
Type	Global ranking	Query-dependent ranking
Idea	“Important pages are linked by important pages”	“Good hubs link to good authorities”
Initialization	Equal	Equal but iterative updates more dynamic
Vulnerability	Link farms	Topic drift



Prepare Below Mention Topics for ETE, questions can be asked related to corresponding Algorithms & Working Examples.

.....

Text Classification:

Naïve Bayes, K-nearest neighbors, Feature selection, Semi-supervised Learning.

Text Clustering:

Vector-space Clustering; K-means, EM algorithm, Text shingling;